Paper Trading Setup — Every Step, Copy-Paste Ready

BEFORE YOU START — Folder Setup

Your project folder is called `fx_trading`. Find where it is on your computer. Open it — it should look like this:

```
fx_trading/
  shared/
  data/
  features/
  research/
  risk/
  state/
  broker/
  execution/
  tests/
  artifacts/      ← may not exist yet, create it
  logs/           ← may not exist yet, create it
  scripts/        ← does not exist yet, you will create it
```

Create two new folders inside `fx_trading/`:

- `fx_trading/artifacts/`
- `fx_trading/logs/`
- `fx_trading/scripts/`

(Right-click → New Folder in Windows Explorer)

STEP 1 — Verify Everything Is Installed

Open **Command Prompt** (press Win+R, type `cmd`, press Enter). Type this and press Enter:

```
python --version
```

Should say `Python 3.12.x`. If it says "not recognized" — Python wasn't added to PATH. Reinstall Python and check the "Add to PATH" box.

Now install all packages. Copy and paste this entire block:

```
pip install pandas numpy scikit-learn lightgbm xgboost statsmodels structlog pandas-ta pytz MetaTrader5
```

STEP 2 — Run the Test Suite

In Command Prompt, navigate to your project folder. Replace `C:\Users\YourName\fx_trading` with your actual path:

```
cd C:\Users\YourName\fx_trading
python -m unittest tests.test_all -v
```

You must see: **48 tests, 0 errors, 0 failures** Do NOT continue if any test fails.

STEP 3 — Write Down Your MT5 Credentials

Open MetaTrader 5. Look at the bottom of the screen — it shows your account number and server name. Or go to **Tools → Options → Server**.

Write these down:

```
Login:    (your 8-digit number)
Password: (the password MT5 gave you)
Server:   MetaQuotes-Demo
```

STEP 4 — Fetch Historical Data

Create a new file: `fx_trading/scripts/fetch_data.py`

Copy and paste this entire file:

```python
```

```python
# scripts/fetch_data.py
# Run this ONCE to download 3 years of price data from MT5.
# Make sure MetaTrader 5 is open and logged in before running.

import sys
import os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from datetime import datetime, timezone, timedelta
from decimal import Decimal
import pickle

import MetaTrader5 as mt5

from shared.schemas import OHLCVBar, Pair, BarSource
from data.aggregation import bars_to_dataframe
from data.time_utils import mt5_server_to_utc

# ── EDIT THIS ──────────────────────────────────────────────────────────────
BROKER_TZ = "Etc/GMT-2"   # MetaQuotes-Demo uses UTC+2
# ───────────────────────────────────────────────────────────────────────────

SYMBOLS = {
    Pair.EURUSD: "EURUSD",
    Pair.GBPUSD: "GBPUSD",
}

DATE_FROM = datetime(2021, 1, 1)
DATE_TO   = datetime(2024, 12, 31)


def convert_mt5_bars(rates, pair: Pair, broker_tz: str) -> list[OHLCVBar]:
    """Convert raw MT5 rates array to OHLCVBar list."""
    result = []
    point = 0.00001 if pair != Pair.USDJPY else 0.001

    for i, r in enumerate(rates):
        # MT5 returns naive datetimes in broker server time
        open_naive = datetime.fromtimestamp(r['time'])
        utc_open   = mt5_server_to_utc(open_naive, broker_tz)
        utc_close  = utc_open + timedelta(seconds=300)

        # MT5 OHLC are mid prices. Approximate bid/ask using spread.
        spread_price = r['spread'] * point
        half = spread_price / 2.0
```

```
        bar = OHLCVBar(
            pair=pair,
            utc_open=utc_open,
            utc_close=utc_close,
            timeframe_sec=300,
            open_bid=Decimal(str(round(r['open'] - half, 6))),
            high_bid=Decimal(str(round(r['high'] - half, 6))),
            low_bid=Decimal(str(round(r['low']  - half, 6))),
            close_bid=Decimal(str(round(r['close'] - half, 6))),
            open_ask=Decimal(str(round(r['open'] + half, 6))),
            high_ask=Decimal(str(round(r['high'] + half, 6))),
            low_ask=Decimal(str(round(r['low']  + half, 6))),
            close_ask=Decimal(str(round(r['close'] + half, 6))),
            volume=int(r['tick_volume']),
            is_complete=True,
            source=BarSource.BROKER_HISTORICAL,
        )
        result.append(bar)

    return result


def main():
    print("Connecting to MetaTrader 5...")
    if not mt5.initialize():
        print(f"ERROR: MT5 initialization failed: {mt5.last_error()}")
        print("Make sure MetaTrader 5 is open and logged in.")
        return

    os.makedirs("data", exist_ok=True)

    for pair, symbol in SYMBOLS.items():
        print(f"\nFetching {symbol}...")

        rates = mt5.copy_rates_range(
            symbol,
            mt5.TIMEFRAME_M5,
            DATE_FROM,
            DATE_TO,
        )

        if rates is None or len(rates) == 0:
            print(f"ERROR: No data returned for {symbol}. Check symbol name.")
            continue

        print(f"  Got {len(rates)} raw bars from MT5")
```

```
        bars = convert_mt5_bars(rates, pair, BROKER_TZ)
        df = bars_to_dataframe(bars)

        # Save as pickle (preserves types exactly)
        out_path = f"data/{symbol.lower()}_5m_bars.pkl"
        with open(out_path, "wb") as f:
            pickle.dump(df, f)

        print(f"  Saved {len(df)} bars → {out_path}")
        print(f"  Date range: {df.index[0]} → {df.index[-1]}")

    mt5.shutdown()
    print("\nDone. Data saved to fx_trading/data/")


if __name__ == "__main__":
    main()
```

Run it:

```
cd C:\Users\YourName\fx_trading
python scripts/fetch_data.py
```

Expected output:

```
Fetching EURUSD...
  Got 262000+ raw bars from MT5
  Saved 262000+ bars → data/eurusd_5m_bars.pkl
```

If it says "MT5 initialization failed" — make sure MetaTrader 5 is open on your screen and logged in.

STEP 5 — Build Labels

Create: `fx_trading/scripts/build_labels.py`

```
python
```

```python
# scripts/build_labels.py
# Constructs trading labels from the historical bar data.

import sys
import os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import pickle
from research.labels import construct_labels, LabelConfig, get_label_distribution

SYMBOL = "eurusd"   # change to "gbpusd" for GBP/USD

def main():
    in_path  = f"data/{SYMBOL}_5m_bars.pkl"
    out_path = f"data/{SYMBOL}_5m_labels.pkl"

    print(f"Loading bars from {in_path}...")
    with open(in_path, "rb") as f:
        df = pickle.load(f)

    print(f"  Loaded {len(df)} bars")
    print(f"  Date range: {df.index[0]} → {df.index[-1]}")

    # Filter out gap-filled bars before labelling
    if "source" in df.columns:
        df = df[df["source"] != "gap_filled"]
        print(f"  After removing gap-filled: {len(df)} bars")

    print("\nConstructing labels...")
    config = LabelConfig(
        tp_atr_multiplier=1.5,
        sl_atr_multiplier=1.0,
        max_holding_bars=12,
        min_atr_pips=3.0,
        max_spread_pips=5.0,
    )

    pair_is_jpy = (SYMBOL == "usdjpy")
    labels = construct_labels(df, config, pair_is_jpy=pair_is_jpy)

    # Print distribution
    dist = get_label_distribution(labels)
    print("\nLabel distribution:")
    print(f"  LONG    : {dist.get('long_pct', 0):.1f}%")
    print(f"  SHORT   : {dist.get('short_pct', 0):.1f}%")
    print(f"  ABSTAIN : {dist.get('abstain_pct', 0):.1f}%")
```

```
    print(f"  Balanced: {dist.get('is_balanced', False)}")

    print("\nHealthy ranges:")
    print("  LONG/SHORT: 20-30% each")
    print("  ABSTAIN:    40-60%")

    # Save
    with open(out_path, "wb") as f:
        pickle.dump(labels, f)
    print(f"\nLabels saved → {out_path}")


if __name__ == "__main__":
    main()
```

Run it:

```
python scripts/build_labels.py
```

Expected output shows LONG ~20-30%, SHORT ~20-30%, ABSTAIN ~40-60%.
If ABSTAIN is above 80%, that's okay for now — continue.

STEP 6 — Fit the Feature Pipeline

Create: `fx_trading/scripts/fit_pipeline.py`

```python
```

```python
# scripts/fit_pipeline.py
# Fits the feature pipeline on TRAINING data only, then saves it.

import sys
import os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import pickle
from features.pipeline import FeaturePipeline, FeaturePipelineConfig

SYMBOL  = "eurusd"
VERSION = "eurusd_5m_v001"

def main():
    in_path  = f"data/{SYMBOL}_5m_bars.pkl"
    out_dir  = f"artifacts/pipeline_{VERSION}"

    print(f"Loading bars from {in_path}...")
    with open(in_path, "rb") as f:
        df = pickle.load(f)

    print(f"  Loaded {len(df)} bars")

    # Use only the first 80% of data to fit the pipeline
    # (never fit on data you'll test on)
    train_end = int(len(df) * 0.80)
    df_train  = df.iloc[:train_end]
    print(f"  Fitting on first {len(df_train)} bars (80% of data)")

    config = FeaturePipelineConfig(
        version="1.0.0",
        rolling_window_bars=100,
        atr_period=14,
        rsi_period=14,
        min_variance_threshold=1e-8,
    )

    pipeline = FeaturePipeline(config)
    pipeline.fit(df_train)

    os.makedirs(out_dir, exist_ok=True)
    pipeline.save(out_dir)
    print(f"\nPipeline saved → {out_dir}")

    # Quick check: transform a small slice
    features = pipeline.transform(df.iloc[:200])
```

```
    print(f"Feature columns ({len(features.columns)}): {list(features.columns[:5])}...")


if __name__ == "__main__":
    main()
```

Run it:

```
python scripts/fit_pipeline.py
```

STEP 7 — Train the Model

Create: `fx_trading/scripts/train_model.py`

```python
```

```python
# scripts/train_model.py
# Runs walk-forward training and saves the final model artifact.
# This will take 10-30 minutes depending on your CPU.

import sys
import os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import pickle
import numpy as np
from features.pipeline import FeaturePipeline
from research.walk_forward import generate_folds, WalkForwardConfig, slice_fold
from research.model_training import (
    ModelConfig, ValidationReport, train_model,
    predict_proba, compute_signal, save_model, seed_everything
)

SYMBOL      = "eurusd"
VERSION     = "eurusd_5m_v001"
PIPELINE_DIR = f"artifacts/pipeline_{VERSION}"

def evaluate_fold(proba, y_test):
    """Compute basic per-fold stats."""
    signals = []
    for i in range(len(proba)):
        sig, conf = compute_signal(proba[i], min_confidence=0.55, min_margin=0.10)
        signals.append((sig, conf))

    n_total    = len(signals)
    n_long     = sum(1 for s, _ in signals if s == 1)
    n_short    = sum(1 for s, _ in signals if s == -1)
    n_abstain  = sum(1 for s, _ in signals if s == 0)

    # Hit rate on non-abstain signals
    correct = 0
    traded  = 0
    for (sig, _), true in zip(signals, y_test.values):
        if sig != 0:
            traded += 1
            if sig == true:
                correct += 1

    hit_rate = correct / traded if traded > 0 else 0.0

    return {
        "n_bars":       n_total,
```

```
        "n_long":       n_long,
        "n_short":      n_short,
        "n_abstain":    n_abstain,
        "abstain_pct":  round(n_abstain / n_total * 100, 1) if n_total > 0 else 0,
        "hit_rate":     round(hit_rate, 3),
        "n_traded":     traded,
    }


def main():
    seed_everything(42)

    # Load data
    print("Loading bars and labels...")
    with open(f"data/{SYMBOL}_5m_bars.pkl", "rb") as f:
        df = pickle.load(f)

    with open(f"data/{SYMBOL}_5m_labels.pkl", "rb") as f:
        labels = pickle.load(f)

    print(f"  Bars: {len(df)}, Labels: {len(labels)}")

    # Load pipeline and transform
    print("Transforming features...")
    pipeline = FeaturePipeline.load(PIPELINE_DIR)
    features = pipeline.transform(df)
    print(f"  Feature matrix: {features.shape}")

    # Align labels to features index
    labels = labels.reindex(features.index)

    # Walk-forward config
    wf_config = WalkForwardConfig(
        train_bars=105_120,
        purge_bars=12,
        val_bars=17_520,
        embargo_bars=12,
        test_bars=8_760,
        step_bars=8_760,
        min_folds=6,
    )

    try:
        folds = generate_folds(len(features), wf_config)
    except ValueError as e:
        print(f"\nERROR: {e}")
        print(f"You have {len(features)} bars but need ~{105_120 + 6*8_760} minimum.")
```

```
    print("Try fetching more data (go back to 2020) in fetch_data.py")
    return

print(f"\nGenerated {len(folds)} walk-forward folds")

# Model config
model_config = ModelConfig(
    version=VERSION,
    pair="EUR/USD",
    timeframe="5m",
    model_class="lightgbm",
    model_params={
        "n_estimators": 500,
        "learning_rate": 0.05,
        "num_leaves": 63,
        "max_depth": -1,
        "min_child_samples": 50,
    }
)

# Train each fold
fold_results = []
final_model  = None

for fold in folds:
    print(f"\nFold {fold.fold_id}/{len(folds)}...")
    X_train, y_train, X_val, y_val, X_test, y_test = slice_fold(features, labels, fold)
    print(f"  Train: {len(X_train)}, Val: {len(X_val)}, Test: {len(X_test)}")

    if len(X_train) < 100 or len(X_val) < 10:
        print("  Skipping — not enough samples")
        continue

    model = train_model(X_train, y_train, X_val, y_val, model_config)
    proba = predict_proba(model, X_test, model_config.model_class)
    result = evaluate_fold(proba, y_test)
    result["fold_id"] = fold.fold_id
    fold_results.append(result)
    final_model = model  # keep the last fold's model

    print(f"  Hit rate: {result['hit_rate']:.1%}  |  "
          f"Abstain: {result['abstain_pct']:.0f}%  |  "
          f"Traded: {result['n_traded']} bars")

if final_model is None:
    print("\nERROR: No folds completed successfully.")
    return
```

```
    # Summary
    print("\n" + "="*50)
    print("FOLD SUMMARY")
    print("="*50)
    avg_hit = np.mean([r["hit_rate"] for r in fold_results])
    avg_abstain = np.mean([r["abstain_pct"] for r in fold_results])
    print(f"Average hit rate : {avg_hit:.1%}")
    print(f"Average abstain  : {avg_abstain:.0f}%")
    print()
    print("Review these numbers. If hit rate is above 55%, that is promising.")
    print("If hit rate is close to 50%, the model may not have learned anything.")

    # Save
    report = ValidationReport(
        fold_results=fold_results,
        aggregate={
            "avg_hit_rate": avg_hit,
            "avg_abstain_pct": avg_abstain,
            "n_folds": len(fold_results),
        },
        deployment_approved=False,  # YOU set this to true after reviewing
    )

    model_config.train_end_date = str(features.index[-1].date())
    artifact_dir, sha256 = save_model(
        final_model, model_config, report, base_path="artifacts/"
    )
    pipeline.save(PIPELINE_DIR)  # re-save to ensure hash is current

    print(f"\nModel saved to: {artifact_dir}")
    print(f"SHA256: {sha256}")
    print()
    print("NEXT STEP: Open the file below and set deployment_approved to true")
    print(f"  {artifact_dir}\\validation_report.json")
    print(f"  {artifact_dir}\\model.manifest.json")


if __name__ == "__main__":
    main()
```

Run it:

```
python scripts/train_model.py
```

This will take **10–30 minutes**. You'll see fold progress printed one by one.

STEP 8 — Approve the Model (Manual Step)

After training finishes, it prints a path like:

```
artifacts/eurusd_5m_v001_20250303/
```

You need to open **two files** in that folder with Notepad and edit them.

File 1: `artifacts/eurusd_5m_v001_20250303/validation_report.json`

Find this line:

```json
"deployment_approved": false
```

Change it to:

```json
"deployment_approved": true
```

Save the file.

File 2: `artifacts/eurusd_5m_v001_20250303/model.manifest.json`

Same change:

```json
"deployment_approved": true
```

Save the file.

> ⚠️ The exact folder name will have today's date in it. Use whatever folder was actually created.

STEP 9 — Start the Demo Trading Loop

Create: `fx_trading/run_demo.py`

```python
```

```python
# run_demo.py
# Starts the live trading loop connected to your MT5 demo account.
# Run this from the fx_trading/ folder.
# MetaTrader 5 must be open and logged in.

import sys
import os
import logging
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

# Set up logging so you can see what's happening
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s  %(levelname)s  %(message)s",
    handlers=[
        logging.StreamHandler(),                    # print to console
        logging.FileHandler("logs/demo.log"),       # save to file
    ]
)

from state.store import StateStore
from broker.mt5_adapter import MT5Adapter
from broker.adapter import IdempotentOrderSubmitter
from features.pipeline import FeaturePipeline
from research.model_training import load_model
from risk.engine import RiskEngine, RiskParameters
from execution.live_loop import LiveExecutionLoop
from shared.schemas import Pair

# ── FILL THESE IN ──────────────────────────────────────────────────────────
LOGIN       = 12345678          # ← your MT5 login number
PASSWORD    = "your_password"   # ← your MT5 password
SERVER      = "MetaQuotes-Demo" # ← your server name
BROKER_TZ   = "Etc/GMT-2"       # ← MetaQuotes-Demo timezone
MODEL_DIR   = "artifacts/eurusd_5m_v001_20250303"  # ← your actual folder name
PIPELINE_DIR = "artifacts/pipeline_eurusd_5m_v001"
# ──────────────────────────────────────────────────────────────────────────

os.makedirs("state", exist_ok=True)
os.makedirs("logs",  exist_ok=True)

print("="*50)
print("FX TRADING BOT — DEMO MODE")
print("="*50)

# 1. State database
```

```
print("Initializing state store...")
store = StateStore("state/trading.db")

# 2. Connect to MT5
print("Connecting to MetaTrader 5...")
adapter = MT5Adapter(
    broker_tz_str=BROKER_TZ,
    login=LOGIN,
    password=PASSWORD,
    server=SERVER,
)

if not adapter.is_connected():
    print("ERROR: Could not connect to MT5.")
    print("Make sure MetaTrader 5 is open and you are logged in.")
    sys.exit(1)

print("MT5 connected OK")

# 3. Load model (will raise if deployment_approved is not true)
print(f"Loading model from {MODEL_DIR}...")
model, manifest = load_model(MODEL_DIR, require_approved=True)
print(f"Model loaded: {manifest['version']} (SHA256: {manifest['sha256'][:12]}...)")

# 4. Load feature pipeline
print(f"Loading pipeline from {PIPELINE_DIR}...")
pipeline = FeaturePipeline.load(PIPELINE_DIR)

# 5. Get pipeline SHA256 for reconciliation
import hashlib, pickle, os as _os
pipeline_path = _os.path.join(PIPELINE_DIR, "pipeline.pkl")
if _os.path.exists(pipeline_path):
    pipeline_sha256 = hashlib.sha256(open(pipeline_path, "rb").read()).hexdigest()
else:
    pipeline_sha256 = manifest["sha256"]  # fallback

# 6. Order submitter
submitter = IdempotentOrderSubmitter(adapter=adapter, state_store=store)

# 7. Risk engine
risk_engine = RiskEngine(RiskParameters(), store)

# 8. Start loop
print("\nStarting live loop. Press Ctrl+C to stop.")
print("Signals will be logged every 5 minutes when a bar closes.\n")

loop = LiveExecutionLoop(
```

```
    adapter=adapter,
    submitter=submitter,
    pipeline=pipeline,
    model=model,
    model_sha256=manifest["sha256"],
    pipeline_sha256=pipeline_sha256,
    model_version=manifest["version"],
    risk_engine=risk_engine,
    state_store=store,
    pairs=[Pair.EURUSD],
)

loop.run(
    model_sha256=manifest["sha256"],
    pipeline_sha256=pipeline_sha256,
)
```

Edit the credentials at the top — replace with your actual login number, password, server name, and the exact folder name from Step 8.

Run it:

```
cd C:\Users\YourName\fx_trading
python run_demo.py
```

STEP 10 — What You Should See

When it's working you'll see log lines every 5 minutes like:

```
2025-03-03 14:05:01  INFO  signal_generated pair=EUR/USD signal=0 confidence=0.512  (ABSTAIN - normal)
2025-03-03 14:10:01  INFO  signal_generated pair=EUR/USD signal=1 confidence=0.621  (LONG signal!)
2025-03-03 14:10:01  INFO  order submitted   pair=EUR/USD side=BUY units=1000
```

In MetaTrader 5, check the **Trade** tab — you'll see positions opening and closing automatically.

`signal=0` (ABSTAIN) is normal. The bot will skip most bars and only trade when it's confident.

TROUBLESHOOTING

Error	Fix
`MT5 initialization failed`	MetaTrader 5 must be **open** on screen before running

Error	Fix
`deployment_approved=false`	Edit both JSON files in Step 8
`ValueError: data too short`	Edit `fetch_data.py` — change `DATE_FROM` to `datetime(2020, 1, 1)` and re-run Steps 4–7
`ModuleNotFoundError: lightgbm`	Run `pip install lightgbm`
`48 tests, X failures`	Do not skip this — fix the failing test
`FileNotFoundError: pipeline.pkl`	Re-run `fit_pipeline.py`
`Login failed`	Double-check your login number, password, and server name in MT5

FILE SUMMARY — What You're Creating and Where

```
fx_trading/
├── scripts/
│   ├── fetch_data.py       ← Step 4
│   ├── build_labels.py     ← Step 5
│   ├── fit_pipeline.py     ← Step 6
│   └── train_model.py      ← Step 7
├── data/
│   ├── eurusd_5m_bars.pkl   ← created by fetch_data.py
│   └── eurusd_5m_labels.pkl ← created by build_labels.py
├── artifacts/
│   ├── pipeline_eurusd_5m_v001/   ← created by fit_pipeline.py
│   └── eurusd_5m_v001_YYYYMMDD/   ← created by train_model.py
│       ├── model.pkl
│       ├── model.manifest.json     ← edit: deployment_approved → true
│       └── validation_report.json  ← edit: deployment_approved → true
├── state/
│   └── trading.db          ← created automatically on first run
├── logs/
│   └── demo.log            ← created automatically on first run
└── run_demo.py             ← Step 9
```